

torch.arange#

<https://pytorch.org/docs/stable/generated/torch.arange.html>

Description:

Returns a 1-D tensor of size $\lceil \frac{\text{end} - \text{start}}{\text{step}} \rceil$ with values from the interval $[\text{start}, \text{end})$ taken with common difference step beginning from start. Note: When using floating-point dtypes (especially reduced precision types like bfloat16), the results may be affected by floating-point rounding behavior. Some values in the sequence might not be exactly representable in certain floating-point formats, which can lead to repeated values or unexpected rounding. For precise sequences, it is recommended to use integer dtypes instead of floating-point dtypes. Note that non-integer step is subject to floating point rounding errors when comparing against end; to avoid inconsistency, we advise subtracting a small epsilon from end in such cases. start (Number, optional) – the starting value for the set of points. Default: 0. end (Number) – the ending value for the set of points

Function Signature

```
torch.arange(start=0, end, step=1, *, out=None, dtype=None, layout=torch.strided, device=None, requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()). If dtype is not given, infer the data type from the other input arguments. If any of start, end, or stop are floating-point, the dtype is inferred to be the default dtype, see get_default_dtype(). Otherwise, the dtype is inferred to be torch.int64. layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor

Code Examples

```
>>> torch.arange(5)
tensor([ 0,  1,  2,  3,  4])
>>> torch.arange(1, 4)
tensor([ 1,  2,  3])
>>> torch.arange(1, 2.5, 0.5)
tensor([ 1.0000,  1.5000,  2.0000])
```

Detailed Documentation

Documentation

Returns a 1-D tensor of size $\lceil \frac{\text{end} - \text{start}}{\text{step}} \rceil$ with values from the interval $[\text{start}, \text{end})$ taken with common difference step beginning from start.

Note: When using floating-point dtypes (especially reduced precision types like `bfloat16`), the results may be affected by floating-point rounding behavior. Some values in the sequence might not be exactly representable in certain floating-point formats, which can lead to repeated values or unexpected rounding. For precise sequences, it is recommended to use integer dtypes instead of floating-point dtypes.

Note that non-integer step is subject to floating point rounding errors when comparing against end; to avoid inconsistency, we advise subtracting a small epsilon from end in such cases.

start (Number, optional) – the starting value for the set of points. Default: 0.

end (Number) – the ending value for the set of points

step (Number, optional) – the gap between each pair of adjacent points. Default: 1.

out (Tensor, optional) – the output tensor.

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). If dtype is not given, infer the data type from the other input arguments. If any of start, end, or stop are floating-point, the dtype is inferred to be the default dtype, see `get_default_dtype()`. Otherwise, the dtype is inferred to be `torch.int64`.

layout (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.